

Словник основних термінів

Прекурс

Біт (bit) — одиниця інформації в комп'ютері представляє собою двійковий розряд, який може приймати значення 0 або 1.

Байт (byte) — одиниця зберігання та передачі даних у комп'ютері. Байт складається з 8 бітів, що дозволяє отримати 256 різних комбінацій 1 і 0.

Система числення — це знакова система, у якій числа записуються за певними правилами за допомогою символів деякого алфавіту, що називаються цифрами і має відповідні правила дій над числами.

Двійкова (або бінарна) система числення — це позиційна система числення з основою 2.

Тема 1. Що таке алгоритми? Вступ до концепції алгоритмів

Алгоритм — це набір інструкцій, які визначають послідовність дій для розв'язання певної задачі або досягнення певного результату.

Оцінка якості алгоритму — це етап у процесі розробки програмного забезпечення, який дозволяє визначити, наскільки ефективно алгоритм розв'язує задачу і чи є можливість його оптимізації.

Найкращий випадок розвитку подій алгоритму — коли для вхідних даних встановленого обсягу потрібна мінімальна кількість операцій виконання алгоритму.

Найгірший випадок розвитку подій алгоритму — коли для вхідних даних встановленого обсягу потрібна максимальна кількість операцій виконання алгоритму.

Середній випадок розвитку подій алгоритму — середня кількість операцій, зазвичай потрібних для опрацювання вхідних даних встановленого обсягу.

Часова складність алгоритму — важливий інструмент для аналізу ефективності алгоритмів, який вимірює, як зростає час виконання алгоритму в залежності від розміру вхідних даних.

Константна складність ($O(1)$) — складність виконання алгоритму, коли незалежно від розміру вхідних даних, алгоритм виконується за сталий час. Приклад: доступ до елемента масиву за його індексом.

Лінійна складність ($O(n)$) — складність виконання алгоритму, коли час виконання алгоритму зростає лінійно відповідно до розміру вхідних даних. Приклад: пошук елемента в несортованому списку.

Логарифмічна складність ($O(\log n)$) — складність виконання алгоритму, коли час виконання алгоритму зростає логарифмічно відносно розміру вхідних даних. Приклад: бінарний пошук у сортованому списку.

Лінійно-логарифмічна складність ($O(n \log n)$) — складність виконання алгоритму, яка зустрічається при сортуванні злиттям або сортуванні швидким способом.

Квадратична складність ($O(n^2)$) — складність виконання алгоритму, коли час виконання алгоритму зростає пропорційно квадрату розміру вхідних даних. Приклад: вкладені цикли, які працюють над всіма парами елементів, такі як алгоритм сортування бульбашкою.

Кубічна складність ($O(n^3)$) — складність виконання алгоритму, коли час виконання алгоритму зростає пропорційно кубу розміру вхідних даних. Прикладом може бути алгоритм, який аналізує всі можливі трійки елементів у масиві.

Експоненціальна складність ($O(2^n)$) — складність виконання алгоритму, коли час виконання алгоритму експоненціально зростає відносно розміру вхідних даних. Приклад: рекурсивне обчислення чисел Фібоначчі.

Факторіальна складність ($O(n!)$) — складність виконання алгоритму, при якій алгоритми часто занадто повільні навіть для невеликих наборів даних. Приклад: задача комівояжера, розв'язана повним перебором.

Просторова складність — інструмент для аналізу ефективності алгоритмів, який вимірює, як об'єм пам'яті, який потрібен для виконання алгоритму, зростає відносно розміру вхідних даних.

Константна просторова складність ($O(1)$) — обсяг пам'яті, необхідний для виконання алгоритму, не залежить від розміру вхідних даних. Наприклад,

алгоритм, який повертає перший елемент списку, має константну просторову складність, незалежно від довжини списку.

Лінійна просторова складність ($O(n)$) — обсяг пам'яті, необхідний для виконання алгоритму, зростає лінійно відносно розміру вхідних даних. Це може стосуватися алгоритмів, які створюють новий список на основі вхідного списку.

Квадратична просторова складність ($O(n^2)$) — ця складність може виникнути, наприклад, при створенні матриць або двовимірних масивів на основі вхідних даних.

Алгоритм Прима — алгоритм пошуку, який використовується для знаходження мінімального кістякового дерева у зваженому незв'язному графі.

Алгоритм Крускала — алгоритм знаходження мінімального кістякового дерева в зваженому графі.

Алгоритм Флойда-Уоршелла — алгоритм, що використовується для знаходження найкоротших шляхів між усіма парами вузлів у графі.

Алгоритм Форда-Беллмана — алгоритм, що дозволяє знайти найкоротший шлях у графі, навіть якщо в ньому є цикли з негативною вагою (але немає циклів з негативною загальною вагою).

Алгоритм максимального потоку — алгоритм пошуку, що допомагає знаходити максимальний потік у мережі потоків, такий як алгоритм Форда-Фалкерсона.

Динамічне програмування — це метод оптимізації, який розв'язує складну задачу, розкладаючи її на менш складні підзадачі. Особливість методу полягає в збереженні результатів підзадач у пам'яті, що забезпечує виконання операцій швидше й дозволяє уникнути повторного розв'язання одних і тих самих підзадач.

Задача про рюкзак — це задача комбінаторної оптимізації: дано речі з певними вагами та вартостями, потрібно вибрати, які речі покласти в рюкзак так, щоб загальна вага не перевищувала вказане обмеження, а загальна вартість була максимальною.

Числа Фібоначчі — це послідовність чисел, у якій кожне наступне число дорівнює сумі двох попередніх чисел. Перші два числа в цій послідовності

завжди дорівнюють 0 і 1. Ось декілька перших чисел цієї послідовності: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34 і так далі.

Тема 2. Основні структури даних

Структури даних — це способи організації та зберігання даних, що дозволяють ефективно виконувати операції над ними. Структури даних важливі для розробки програмного забезпечення, оскільки вони допомагають зберігати та організувати дані в комп'ютері. Ці структури включають масиви, списки, стеки, черги, хеш-таблиці, дерева, граfi та інше.

Масиви (Arrays) — це структури даних, що використовуються для зберігання однотипних елементів у пам'яті. Масиви є особливо ефективними при обробці великих обсягів даних, таких як зображення, звуки, числові датасети тощо.

Словники (Dictionaries) — це структури даних, засновані на принципі ключ-значення. Вони використовуються для швидкого доступу до даних за ключем, для зберігання метаданих (дані про дані), для зберігання налаштувань програм, для роботи з JSON-даними тощо. Словник у Python — це неупорядкована колекція елементів, яка зберігає дані у форматі ключ-значення.

Множини (Sets) — це структури даних, які використовуються для зберігання унікальних неупорядкованих елементів. Множини використовуються для виконання математичних операцій, таких як об'єднання, перетин, різниця, а також для швидкого виявлення унікальності елементів. Множини в Python — це неупорядковані колекції унікальних елементів.

Стек — це структура даних, у якій доступ до елементів обмежений таким чином, що лише останній доданий елемент можна видалити (цей принцип називається LIFO — *Last In, First Out*).

Черги — це структура даних, де першим є елемент, який був доданий першим. Це також відомо як FIFO (*First In First Out*).

Двостороння черга, або просто **"deque"** (від англ. "double-ended queue") — особлива структура даних, яка дозволяє додавати та видаляти елементи на обох кінцях. Відмінність deque від звичайних списків полягає в тому, що вона оптимізована для вставок та видалень на обох кінцях.

Списки зв'язаних елементів, або зв'язні списки (Linked Lists) — це структура даних, яка складається з вузлів. Кожен вузол містить елемент даних і вказівник (зв'язок) до наступного вузла у списку.

Хеш-таблиці — це структура даних, яка реалізує інтерфейс асоціативного масиву і використовує хеш-функцію для конвертації ключів в індекси масиву, у якому зберігаються дані. У Python цю структуру реалізовано через словники (`dict`).

Хеш-функція — це функція, яка перетворює вхідні дані довільного розміру на вихідні дані фіксованого розміру. Вихідні дані, відомі як хеш-код, зазвичай являють собою короткий ланцюжок символів, які можуть слугувати як ключ у хеш-таблиці.

Тема 3. Рекурсивні функції, алгоритми та приклади їх застосування

Рекурсія — це концепція в програмуванні, коли функція викликає саму себе в рамках власного виконання. Це схоже на дроблення великої проблеми на менші, керованіші проблеми.

Базовий випадок рекурсії — це умова, при якій рекурсія припиняє виклик самої себе, щоб уникнути нескінченного циклу. Він важливий для запобігання нескінченному циклу в рекурсії.

Рекурсивний випадок — це умова, за якої функція викликає саму себе з новими аргументами. Фактично це ситуація, коли функція продовжує викликати саму себе, розбиваючи проблему на менші частини.

Стек викликів рекурсії — це специфічна частина пам'яті, яка використовується для зберігання інформації про активні виклики функцій. Кожен раз, коли функція викликається, створюється новий запис (або "шар") у цьому стеку для цього конкретного виклику.

Ітерація — це спосіб повторення операцій у програмуванні. Ітерація здійснюється за допомогою циклічних структур, таких як цикли `for` і `while`.

Мемоізація — це метод оптимізації, який дозволяє зберігати результати викликів функцій, щоб пізніше уникнути повторних викликів з тими самими параметрами. Вона часто використовується для оптимізації рекурсивних алгоритмів, де одних і тих самих викликів може бути багато.

Рекурсивний алгоритм — це алгоритм, який розв'язує задачу, розбиваючи її на підзадачі того самого типу, але меншого розміру, і викликаючи самого себе для їх розв'язання. Рекурсивний алгоритм складається з базового випадку, що представляє найпростішу форму задачі, і рекурсивного випадку, який формулюється через виклики самого себе.

Фрактали — це геометричні об'єкти, які характеризуються самоподібністю на різних масштабах. Це означає, що частина фракталу при досить сильному збільшенні виглядає так само, як і весь фрактал.

Обхід дерева каталогів за допомогою рекурсії — це підхід, коли програма систематично "відвідує" всі підкаталоги та файли в директорії, використовуючи рекурсивний алгоритм.

Тема 4. Алгоритми сортування

Сортування — це процес впорядкування набору даних за певними критеріями. Принципи сортування можуть бути різними, але в основі всіх них лежить ідея порівняння двох елементів і перестановки їх місцями, якщо вони не відповідають заданому порядку.

Алгоритми сортування — це специфічний клас алгоритмів, що спрямовані на впорядкування даних.

Bubble sort (Метод "бульбашки", алгоритм прямого обміну) — це один із найпростіших методів сортування. Його основна ідея полягає в порівнянні сусідніх елементів і перестановці їх місцями, якщо вони розташовані в неправильному порядку. Процес повторюється, доки весь масив не буде впорядкований.

Insertion sort (Метод вставок) — метод сортування, у якому елементи масиву перебираються по одному, і кожний наступний елемент вставляється на відповідне місце в середині вже відсортованої частини масиву.

Quicksort (Швидке сортування) — алгоритм сортування, який використовує стратегію "розділяй та володарюй". Він вибирає один з елементів масиву як "опорний" і організовує решту масиву навколо нього таким чином, що елементи, менші за опорний, знаходяться зліва від нього, а більші — справа.

Сортування прямим вибором (Алгоритм прямого вибору (Selection Sort)) — це метод, який працює, систематично вибираючи найменший (або найбільший, залежно від порядку сортування) елемент із невідсортованої

частини списку та міняючи його місцями з першим невідсортованим елементом. Назва алгоритму вказує на його головний принцип: вибір найбільш відповідного елемента на кожному кроці.

Сортування злиттям — ще один ефективний метод сортування, який використовує принцип "розділяй та володарюй". Він розбиває масив на дві половини, сортує їх окремо, а потім зливає їх, щоб отримати відсортований масив. Це забезпечує надійну й стабільну процедуру сортування з гарантованою часовою складністю $O(n \log n)$.

Алгоритм сортування Шелла — вдосконалення методу сортування вставками. Основна ідея полягає в тому, щоб сортувати елементи на певних інтервалах, замість порівнювання кожного елемента зі своїм сусідом. При кожній ітерації цей інтервал зменшується, алгоритм наближається до сортування вставками, коли інтервал дорівнює 1.

Алгоритм порозрядного сортування (Radix Sort) — це сортування, яке впорядковує числа за кожним розрядом почергово, від найменшого до найбільшого розряду. Це нелінійний алгоритм сортування, який здійснює сортування на основі цифр числа.

Тема 5. Алгоритми пошуку

Алгоритми пошуку — алгоритми, які спрямовані на виявлення конкретних елементів у великих наборах даних.

Лінійний пошук — один із найбільш базових алгоритмів пошуку, який складається в простому послідовному перегляді елементів масиву. Незважаючи на свою простоту, він може бути неефективним на великих наборах даних.

Бінарний пошук — це витончений метод пошуку, який вимагає попереднього сортування масиву. Завдяки систематичному розділенню масиву навпіл і виключенню половини даних на кожному кроці, цей алгоритм може швидко знаходити елементи, забезпечуючи значне прискорення, порівняно з лінійним пошуком.

Інтерполяційний пошук — цей метод пошуку схожий на бінарний пошук, але він робить припущення про розподіл ключів у масиві. Це алгоритм пошуку, який використовує ідею інтерполяції для пошуку елемента у відсортованому списку або масиві. Він працює швидше, ніж бінарний пошук для списків з великим розмахом значень, де ключі розподілені рівномірно.

Пошук за допомогою хешування — метод пошуку за допомогою хеш-таблиць, які дозволяють шукати елементи за сталою часовою складністю $O(1)$ у середньому випадку, використовуючи специфічну функцію хешування для прямого звертання до потрібного місця в таблиці.

Індексно-послідовний пошук (Indexed Sequential Search) — це метод пошуку, який комбінує переваги послідовного та двійкового пошуку. Цей алгоритм являє собою гібридний метод, який об'єднує в собі переваги послідовного та двійкового пошуку. Його основна ідея полягає в тому, щоб використовувати індексну таблицю для визначення діапазону, де знаходиться шуканий елемент, а далі застосовувати послідовний пошук у цьому діапазоні.

Індексна таблиця — це просто підмножина основного масиву, яка використовується для швидкого знаходження діапазону, в якому може знаходитися елемент пошуку.

Колізія — це проблема, яка існує у хеш-таблицях. Вона виникає тоді, коли два різні ключі продукують одне й те ж саме значення хешу. Це призводить до конфлікту, оскільки хеш-таблиці призначено зберігати лише одне значення для кожного індексу.

Лінійне зондування (Linear probing) — це метод розв'язання колізії, коли програма починає шукати таблицю зліва направо, поки не знайде вільну комірку. Це працює, але може призвести до "кластеризації", коли велика кількість колізій зводить нанівець переваги швидкого пошуку, який надають хеш-таблиці.

Метод ланцюжків (Chaining) — це метод розв'язання колізії, згідно якого якщо два різні ключі мають однаковий хеш, вони об'єднуються в один "ланцюг" (список або іншу структуру даних), що зберігається в одній і тій самій позиції в хеш-таблиці. При пошуку значення за ключем ми перш за все знаходимо позицію в хеш-таблиці за допомогою хеш-функції, а потім шукаємо відповідне значення в ланцюгу.

Бінарні дерева пошуку — дерева, які використовуються, коли дані повинні бути впорядковані. Вони також дозволяють швидкий пошук, вставку та видалення, але вимагають більше пам'яті, ніж хеш-таблиці.

В-дерева — це дерева, які використовуються в системах, де дані зберігаються на зовнішніх пристроях, таких як диски.

Блумові фільтри — це просторо-ефективна структура даних, яка дозволяє визначити, чи є елемент у наборі, і використовує декілька хеш-функцій.

Наївний алгоритм пошуку підрядка в рядку — це один з найпростіших способів знаходження підрядка в головному рядку. Він полягає в послідовному порівнянні підрядка з кожним відрізком головного рядка.

Алгоритм Кнута-Морріса-Пратта (КМП) — це ефективний алгоритм пошуку підрядка в рядку, що працює за лінійний час. Основна ідея алгоритму полягає в тому, щоб при порівнянні рядків уникати перевірки символів, які в процесі вже були порівняні. Для цього алгоритм створює префіксну функцію.

Алгоритм Боєра-Мура — це алгоритм пошуку підрядка, який оптимізований для сканування тексту справа наліво. Ідея алгоритму полягає в тому, що коли відбувається порівняння зразка з текстом та при цьому останній символ зразка не збігається з поточним символом тексту, ми можемо безпечно змістити зразок далі, залежно від місця останнього входження даного символу в зразок.

Алгоритм Рабіна-Карпа — це алгоритм для пошуку підрядка (або патерну) в рядку тексту за допомогою хешування. Він використовує віконний підхід, де розмір вікна відповідає розміру патерну. Алгоритм особливо ефективний для наборів даних, де є багато можливих входжень патерну.

Тема 6. Графи

Граф — це набір об'єктів, які називаються вершинами (або вузлами), що можуть бути пов'язані лініями, які називають ребра. Це математична структура, яка використовується для моделювання паросполучених об'єктів.

Вершина — це елемент графа, який представляє об'єкт. Є основним об'єктом у графі. Вона може представляти будь-яку сутність залежно від контексту.

Вузли в графі — це ключові точки, які можуть бути з'єднані ребрами.

Ребро — це елемент графа, який представляє зв'язок між двома вершинами. Ребра можуть бути **напрямленими** або **ненапрямленими**. Направлене ребро вказує на напрямок від однієї вершини до іншої, тоді як ненаправлене ребро просто з'єднує дві вершини без вказівки на напрямок.

Ступінь вершини (degree of vertex) — це число її сусідів, це кількість ребер, які виходять з вершини.

Сусіди вершини — це вершини, з якими вершина пов'язана ребрами. Якщо дві вершини з'єднані ребром, то ми говоримо, що вони **суміжні (adjacent)**. Усі суміжні вершини називаються **сусідніми (neighbors)**.

Кількість ребер — це загальна кількість ребер у графі.

Орієнтований граф (орграф) — це граф, у якому ребра мають напрямок.

Неорієнтований граф (undirected graph) — це граф, у якому порядок вершин не має значення. Це граф, у якому ребра не мають напрямку.

Розріджений граф — це граф, у якому мало ребер.

Щільний граф — це граф, у якому багато ребер.

Петлі (self-loops) — ребра, які починаються і закінчуються в одній і тій самій вершині

Простий граф (simple graph) — це граф без петель і паралельних ребер.

Матриця суміжності — це квадратна матриця, у якій кожен елемент матриці представляє наявність або відсутність ребра між двома вершинами. Якщо між двома вершинами є ребро, то відповідний елемент матриці дорівнює 1, інакше він дорівнює 0.

Списки суміжності — це структура даних, яка зберігає список вершин, з якими пов'язана кожна вершина. Список суміжності для цього графа можна представити у форматі словника.

Матриця інцидентності — це ще один спосіб представлення графа. Стовпці матриці відповідають ребрам, рядки — вершинам. Ненульове значення в клітинці матриці вказує на зв'язок між вершиною і ребром (їх інцидентність).

Пошук у ширину (Breadth-First Search — BFS) — це алгоритм для обходу або пошуку дерева або графу. BFS розпочинає обхід з кореневого вузла та відвідує вузли в порядку їх ширини.

Пошук у глибину (Depth-First Search — DFS) — це алгоритм для обходу або пошуку дерева або графу. DFS використовує стекову структуру даних для ефективного обходу вузлів.

Циклічний граф — це граф, який характеризується наявністю одного або більше циклів, тобто шляхів, які починаються та закінчуються в одній і тій самій вершині. Це основна відмінність циклічних графів від ациклічних графів, які не містять жодного циклу.

Зв'язний граф — це граф, у якому існує шлях між кожною парою вершин. Це означає, що ви можете дійти від будь-якої вершини до будь-якої іншої вершини графа вздовж цього ребер графа.

Ступінь центральності (Degree Centrality) — це як кількість з'єднань, які має вузол. Чим більше з'єднань має вузол, тим більш центральним він є. У ненаправлених графах це просто кількість сусідів вузла. У направлених графах можна розглядати вхідний та вихідний ступені окремою.

Близькість вузла (Closeness Centrality) — це обернене значення середньої відстані від вузла до всіх інших вузлів у графі. Вузли, які знаходяться ближче до інших вузлів, мають вищу центральність близькості.

Посередництво вузла (Betweenness Centrality) — це кількість найкоротших шляхів між усіма парами вузлів, які проходять через даний вузол. Ця метрика відображає, наскільки вузол є "мостом" між іншими вузлами у графі.

Повний граф — це специфічний тип графа, в якому кожен вузол пов'язаний з кожним іншим вузлом. Іншими словами, між будь-якою парою вузлів існує ребро.

Алгоритм Дейкстри — цей алгоритм пошуку, який знаходить найкоротший шлях від одного початкового вузла до всіх інших вузлів у зваженому графі.

Алгоритм PageRank — це алгоритм, розроблений для оцінки важливості веб-сторінок на основі гіперпосилань між ними. Основна ідея полягає у тому, що значущість веб-сторінки можна визначити за кількістю посилань на неї з інших веб-сторінок, а також за важливістю цих веб-сторінок.

Тема 7. Дерева та балансування

Дерева — специфічний тип графа, у якому жодні дві вершини не з'єднані більше ніж одним шляхом. Це підтип графів, який містить у центрі початковий вузол, який називають корінь. Від цього вузла можуть відходити різні гілки, які, у свою чергу, закінчуються іншими вузлами. Ці вузли можуть мати власні підвузли, і так далі, утворюючи ієрархічну структуру. Дерево складається з **вузлів**, де кожен вузол може мати декілька **дочірніх вузлів**, але лише один **батьківський вузол**.

Корінь дерева — це вершина дерева, яка не має батьківського вузла

Батьківський вузол — це вузол з одними більш дочірніми вузлами. У братніх вузлів є спільний батьківський вузол.

Ребро — це з'єднання між двома вузлами в дереві.

Лист — це вузол без дочірніх вузлів.

Вузол розгалуження — це вузол із дочірніми вузлами.

Обхід дерева — це процес відвідування всіх вузлів дерева та виконання операції (зазвичай виводу або іншої обробки) на кожному вузлі. Головна мета обходу — відвідати кожен вузол дерева рівно один раз. Залежно від послідовності, в якій ми відвідуємо вузли, існують різні способи обходу.

Двійкове дерево пошуку (ДДП) — це деревоподібна структура даних, що має наступні властивості:

1. Кожен вузол має ключ.
2. Ключі в лівому піддереві будь-якого вузла завжди менші за його ключ.
3. Ключі в правому піддереві будь-якого вузла завжди більші за його ключ.
4. Ліве та праве піддерева також є ДДП.

Зважені збалансовані дерева — це різновид дерев двійкового пошуку, які додають концепцію ваги до кожного вузла з метою підтримання балансу. Вони забезпечують, що дерево залишається збалансованим під час операцій вставки та видалення з метою забезпечення оптимальної швидкості виконання операцій.

Збалансовані за висотою дерева (AVL-дерева) — це різновид дерев двійкового пошуку, у яких балансування здійснюється на основі висоти піддерев. Вони гарантують, що висота дерева залишається логарифмічною відносно кількості вузлів, тобто $O(\log n)$, де n - кількість вузлів у дереві.

Червоно-чорні дерева (RBT) — це один з найпопулярніших видів збалансованих бінарних дерев пошуку, відомих своєю здатністю до швидкого самобалансування. Вони забезпечують ефективне виконання операцій пошуку, вставки та видалення, навіть у випадках із великою кількістю даних.

2-3-дерева — це ще один вид збалансованих бінарних дерев пошуку, які відрізняються своєю унікальною структурою та методами балансування. Вони дозволяють мати вузли з двома або трьома дітьми, що надає їм додаткову гнучкість у балансуванні, та забезпечують стабільно високу

продуктивність для операцій пошуку, вставки та видалення, адаптуючись до змін у даних на льоту.

АА-дерева — це варіант червоно-чорних дерев, але зі спрощеними правилами для балансування. Вони забезпечують швидкий доступ до даних, ефективні операції вставки та видалення, і все це з меншою кількістю обертань.

В-дерева — це варіант бінарних дерев, що відрізняються тим, що вони спеціально розроблені для ефективної роботи з великими обсягами даних, які зберігаються на диску або в іншій постійній пам'яті. Ці дерева мають високу ступінь гілкування, що дозволяє зменшити кількість операцій введення-виведення при роботі з даними.

Тема 8. Купи, або піраміди

Черга з пріоритетами — це абстрактна структура даних, яка дозволяє додавати об'єкти в колекцію в довільному порядку, але видаляти їх за пріоритетом, а не за принципом "перший прийшов — перший обслугований", як в звичайних чергах.

Купа (або піраміда) — це спеціальний вид бінарного дерева. Основна властивість купи полягає в тому, що значення кожного вузла має перевищувати значення кожного з його нащадків.

Максимальна купа (Max Heap) — купа, ключ кожного вузла якої більший або дорівнює ключам його дітей. Корінь максимальної купи містить найбільший елемент.

Мінімальна купа (Min Heap) — купа, ключ кожного вузла якої менший або дорівнює ключам його дітей. Корінь мінімальної купи містить найменший елемент.

Побудова купи (heapifying) — створення купи з такої структури даних, як масив.

Пірамідальне сортування (Heap Sort) — це метод сортування, що використовує структуру даних піраміда (heap) для сортування елементів за зростанням чи спаданням.

Тема 9. Жадібні алгоритми та динамічне програмування

Жадібні алгоритми — це клас алгоритмів оптимізації, які у кожному кроці вирішують підзадачу, вибираючи найкращий локальний варіант з надією знайти глобальний оптимум.

Динамічне програмування — це метод розв'язання складних задач шляхом розбиття їх на простіші підзадачі. Він використовується, коли задача має підзадачі, що перекриваються, тобто одні й ті самі дрібніші задачі розв'язуються знову і знову в процесі обчислення.

Алгоритм Флойда-Воршала — це алгоритм пошуку найкоротших шляхів між усіма парами вершин в зваженому графі. Він працює для графів з від'ємними вагами ребер, але не працює, якщо в графі є цикли з від'ємною сумарною вагою (такі цикли створюють проблему, оскільки можна безкінечно обертатися по циклу і зменшувати відстань).

Задача про рюкзак — це класична проблема оптимізації в області комп'ютерних наук. Існує кілька варіантів цієї задачі, але найпоширенішим є задача пакування рюкзака: "У вас є рюкзак з обмеженою місткістю та набір предметів, кожен з яких має певну масу та вартість".

Брутфорс (Brute-force) — це метод повного перебору. В задачі про рюкзак, наприклад, необхідно перевірити всі можливі комбінації предметів, щоб знайти найкращий набір.

Задача комівояжера (TSP – Traveling Salesman Problem) — це класична задача оптимізації, яка полягає в знаходженні найкоротшого можливого маршруту, який відвідає кожне місто лише один раз і повернеться до початкового міста. Це NP-повна задача, що означає, що час знаходження точного рішення зростає експоненційно зі збільшенням кількості міст.

Алгоритм Хелда-Карпа для задачі комівояжера — це метод динамічного програмування, що значно ефективніше повного перебору, особливо для середніх і невеликих розмірів задач.

Мінімальне остовне дерево для зваженого неорієнтованого графа — це підмножина ребер цього графа, яка формує дерево, включає всі вершини і має найменшу можливу сумарну вагу всіх ребер.

Алгоритм Краскала — це жадібний алгоритм, який використовується для знаходження мінімального остовного дерева в зваженому неорієнтованому графі.

Алгоритм Прима — це один жадібний алгоритм, який також використовується для знаходження мінімального остовного дерева для зваженого неорієнтованого графа.

Тема 10. Лінійне програмування та рандомізовані алгоритми

Лінійне програмування — це метод оптимізації, призначений для максимізації або мінімізації лінійної цільової функції, підкоряючись набору лінійних рівностей або нерівностей, які формують обмеження.

Цільова функція — це лінійне рівняння, яке потрібно максимізувати або мінімізувати.

Обмеження — це умови, які повинні бути дотримані вашими змінними.

Рандомізовані алгоритми — це клас алгоритмів, які використовують випадковість під час своєї роботи. Вони можуть використовувати випадкові числа або випадкові вхідні дані для прийняття рішень, і, як наслідок, їхній вихід може варіюватися при кожному виконанні.

Алгоритм Рабіна-Міллера — це пробабілістичний тест простоти, який використовується для визначення, чи є дане число простим.

Універсальне хешування — це метод, при якому вибирається хеш-функція випадковим чином з сімейства хеш-функцій, що гарантує, що для будь-якого набору ключів ймовірність колізій буде мінімальною.

Метод Монте-Карло — це статистичний алгоритм, який використовує випадковість та імовірність для вирішення різних проблем, які можуть бути складними або неможливими для точного розв'язання традиційними методами. Основна ідея полягає в тому, щоб використовувати випадкові зразки для апроксимації результатів.